

TECHNICAL REPORT · 2026

ms-swift 大模型训练框架 全景与源码级指南

从 0 到 1 建立训练框架心智模型，再深入掌握 CLI、数据、模板、Trainer、RL 后训练、Megatron 并行、评测部署与扩展机制。

Winston

V1.0 · 2026-05-19

资料口径: ms-swift main commit 7dd6b0e; PyPI / GitHub 最新稳定版 v4.2.1

目录

01	执行摘要：先建立正确的框架观	全局
02	定位与边界：ms-swift 解决哪一类训练问题	认知
03	整体架构：从命令入口到权重产物	架构
04	命令与参数：一次运行如何被解释	入口
05	数据、模板与损失：训练样本如何变成梯度	核心
06	CPT / SFT 主链路：最常用训练路径	实践
07	轻量微调与显存工程：LoRA、QLoRA、packing	效率
08	RL 后训练：DPO、RM、GRPO、GKD 与 rollout	对齐
09	Megatron-SWIFT：大模型并行训练路线	规模
10	多模态、Agent 与专门任务	任务
11	评测、推理、导出与部署	闭环
12	扩展、排障与掌控清单	进阶

执行摘要：先建立正确的框架观

ms-swift 不是一个只包了一层训练命令的 LoRA 脚手架，而是 ModelScope 社区维护的**大模型与多模态大模型训练到部署全链路框架**。截至本稿核实时，官方 README 与 PyPI 页面宣称其支持 600+ 纯文本大模型、400+ 多模态大模型、150+ 数据集，并覆盖预训练、监督微调、人类对齐、推理、评测、量化和部署。

要真正掌握 ms-swift，不能把注意力停在一条 ``swift sft`` 命令上。正确的读法是把它拆成七层：命令入口负责把 CLI、YAML、分布式环境变量转成可执行进程；参数系统把模型、数据、模板、训练、生成、量化和后端选择统一起来；模型与模板注册层决定如何加载权重、如何把 messages 编码成 token；数据层决定样本清洗、列映射、packing、lazy tokenize 与缓存策略；Trainer 层把任务选择映射到 Hugging Face Trainer、TRL Trainer 或 ms-swift 自己的混入逻辑；RL 与 rollout 层处理采样、奖励、参考模型、vLLM 服务器和权重同步；导出与推理层把 checkpoint 重新接回实际服务。

五个核心结论

- ms-swift 的核心抽象不是模型，而是“模型元信息 + 数据标准格式 + 对话模板 + Trainer”。模型能否接入，取决于 ``model_type``、``template``、`loader`、`model_arch` 与额外保存文件是否定义完整。
- 训练命令只是入口，真正的主链路在 ``swift/pipelines``。``swift sft`` 会进入 ``SwiftSft`` pipeline，完成模型加载、模板准备、数据加载和编码、tuner 包装、TrainerFactory 选择、训练与状态保存。
- 数据是最容易被低估的控制面。同一个 JSONL 文件可以通过 ``messages``、``rejected_response``、``label``、``images``、``tools``、``loss``、``loss_scale`` 等字段进入 SFT、DPO、KTO、GRPO、Agent、多模态训练等不同任务。
- RL 后训练的复杂度来自“生成”和“训练”的耦合。GRPO 不只是在 Trainer 里加一个损失函数，它需要 rollout、奖励函数、组内优势估计、参考策略 KL、vLLM / SGLang / LMDeploy 等推理后端以及权重同步共同工作。
- Megatron-SWIFT 是规模化路线，不是初学者默认路线。普通 SFT 优先用 ``swift sft``；MoE、长上下文、FP8、TP/PP/CP/EP/VPP 或多机大规模训练，再切到 ``megatron sft/r1hf/export`` 与 Mcore-Bridge。

本文读法

第一遍按章节顺序读，建立整体逻辑；第二遍只看图表和决策表，形成操作路线；第三遍对照源码路径和命令清单，把自己的训练任务映射到具体参数、数据格式和扩展点。

资料口径与严谨边界

项目	本稿采用口径	说明
稳定版本	PyPI `ms-swift 4.2.1`, 发布于 2026-05-17	用于安装、版本判断和稳定能力描述。
主分支源码	commit `7dd6b0e`, 2026-05-19 19:30 +0800	用于目录结构、pipeline、Trainer、RL trainer 等源码级解读。
官方文档	ReadTheDocs latest 标示为 `swift 4.3.0.dev0`	本文把 dev 文档中的新增能力与稳定 v4.2.1 区分表达。
论文口径	arXiv 2408.05517 v4; AAAI 2025 Demonstration Track	用于框架设计动机和学术引用, 不用论文中过期数量覆盖 README 当前数量。

定位与边界：ms-swift 解决哪一类训练问题

ms-swift 的定位可以简化为一句话：让研究者和工程师以统一接口完成大模型与多模态大模型的训练、对齐、评测、量化、推理和部署。它的优势不是某一个单点算法，而是把模型适配、数据格式、训练策略、推理后端和导出链路收束成一个可复用系统。

它站在什么层次

训练框架通常分四层。最底层是 PyTorch、NCCL、CUDA、NPU runtime 等执行基座；再上一层是 Transformers、PEFT、TRL、DeepSpeed、FSDP、Megatron、vLLM 等通用库；第三层是面向 LLM / MLLM 的模型、数据、模板、任务协议；最上层才是用户直接操作的 CLI、Web-UI、Python API 与脚本。ms-swift 主要工作在第三层和第四层，同时把第二层的能力通过参数暴露出来。

FIGURE 1 MS-SWIFT STACK POSITION



ms-swift 位于大模型训练的任务协议层，它把底层库的自由度约束成可复用的训练流程。

能力边界

能力域	ms-swift 覆盖什么	不应误解为什么
训练任务	CPT、SFT、GRPO、GKD、DPO、KTO、RM、CPO、SimPO、ORPO、PPO、Embedding、Reranker、序列分类。	不是自动给出最优算法；算法选择仍依赖数据、目标和预算。
模型支持	通过模型注册与模板注册适配大量 LLM / MLLM，并默认使用 ModelScope 下载，也可切换 Hugging Face。	不是所有模型天然可训；新模型仍要核对 loader、template、model_arch 和特殊参数。
分布式与显存	DDP、device_map、DeepSpeed ZeRO、FSDP/FSDP2、sequence parallel、Megatron 并行、packing、padding_free、gradient checkpointing。	不是“一键省显存”；每个技术都会改变吞吐、稳定性或导出方式。
推理部署	Transformers、vLLM、SGLang、LMDeploy 推理后端，OpenAI 风格部署接口，LoRA merge、量化导出。	不是专门的推理服务平台；生产服务仍需监控、鉴权、限流与灰度。
扩展	模型、数据集、模板、loss、loss_scale、metrics、callbacks、optimizer、tuner、ORM、PRM、Agent template。	不是所有扩展都适合外部插件；进入主仓库前需要测试对齐和文档。

判断是否使用 ms-swift 的简单标准：如果你的工作是“把一个开源或自定义 LLM / MLLM 在特定数据上完成可复现训练，并继续评测、导出、部署”，ms-swift 是高匹配；如果你要重写底层并行内核、研究全新的优化器理论，ms-swift 更适合作为实验承载层，而不是研究对象本身。

学习路线

1. 先跑通单卡 LoRA SFT，理解 `model`、`dataset`、`template`、`output\_dir`、`adapters`、`merge\_lora` 这些对象的生命周期。
2. 再读 `swift/pipelines/train/sft.py`，把命令行参数和源码对象一一对应起来。
3. 随后学习标准数据集格式，尤其是 `messages`、`rejected\_response`、`label`、`images`、`tools`、`loss`、`loss\_scale`。
4. 进入 RL 后训练前，先能解释 DPO / KTO / RM / GRPO 的数据差异，不要先调 reward function。
5. 需要大模型并行时再读 Megatron-SWIFT，否则过早学习 TP、PP、CP、EP 会掩盖主链路。

整体架构：从命令入口到权重产物

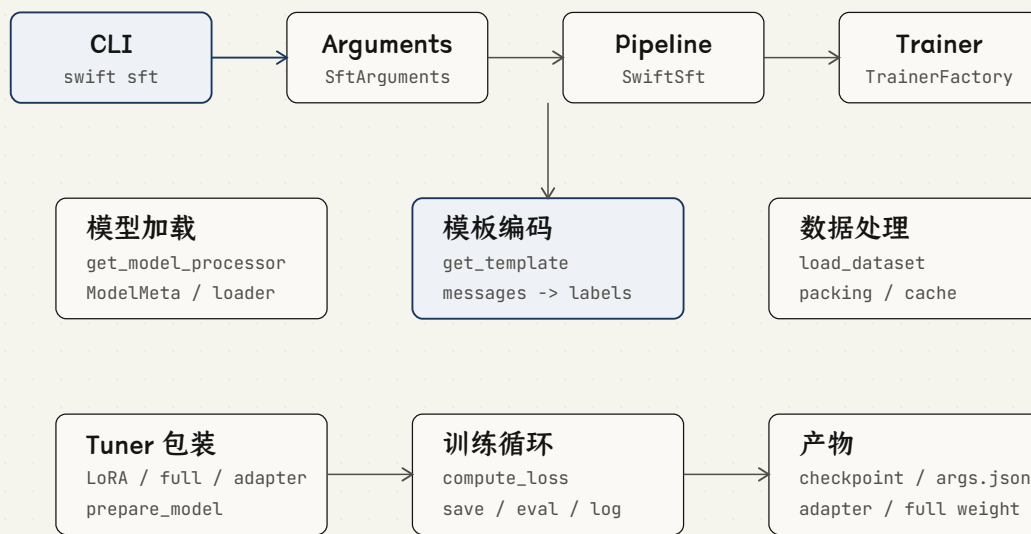
ms-swift 的源码结构非常直接：一级目录基本对应一类扩展点或运行阶段。掌握这些目录，比记住几十个参数更重要，因为参数最终都会落到这些对象上。

源码目录地图

目录	职责	掌握重点
`swift/cli`	命令分发与进程启动。	`swift sft`、`swift rlhf`、`swift infer` 等命令如何映射到 pipeline；何时自动走 torchrun。
`swift/arguments`	参数 dataclass、参数校验、从 checkpoint 读取 args。	`model`、`dataset`、`template`、`training_args` 如何被组合。
`swift/model`	模型元信息、模型加载、模型注册、NPU patch。	`ModelMeta`、`MODEL_MAPPING`、loader、model_arch。
`swift/template`	对话模板、多模态 token、messages 到 input_ids / labels 的编码。	训练哪些 token、忽略哪些 token、系统提示优先级。
`swift/dataset`	数据加载、列映射、标准格式转换、packing、缓存数据集。	`AutoPreprocessor`、`LazyLLMDataset`、`PackingDataset`、streaming。
`swift/pipelines`	训练、推理、评测、导出、采样等主流程。	读源码先读 pipeline，再读 trainer。
`swift/trainers`	SFT、预训练、Embedding、Reranker、序列分类等 Trainer。	基于 HF Trainer 的混入逻辑、loss、accuracy、dataloader。
`swift/rlhf_trainers`	DPO、KTO、RM、PPO、GRPO、GKD 等后训练 Trainer。	与 TRL 的关系、rollout、ref model、奖励、vLLM 客户端。
`swift/rollout` / `swift/rewards`	RL 采样和奖励函数。	GRPO / GKD 何时需要外部推理服务，ORM / PRM 如何挂入。
`swift/megatron`	Megatron-SWIFT 参数、pipeline、trainer、模型转换。	TP、PP、SP、CP、EP、Mcore-Bridge 与 checkpoint 格式。
`swift/infer_engine`	Transformers、vLLM、SGLang、LMDeploy 等推理后端。	训练后验证、批量推理、部署和 RL rollout 共享推理能力。
`swift/ui`	Web-UI 的训练、推理、评测、量化界面。	适合低门槛试验，不替代脚本化生产流程。

一次 SFT 运行的架构流

FIGURE 2 SFT EXECUTION PATH



`swift sft` 的运行路径不是线性脚本，而是参数、模型、模板、数据、tuner 与 Trainer 的对象协作。

为什么说 pipeline 是源码阅读入口

初学者容易先读 `Trainer`，但这会丢失上下文。Trainer 只知道如何训练给定模型与数据集，不知道模型如何下载、模板如何选择、数据何时 lazy encode、LoRA 何时注入、checkpoint 如何附带 `args.json`。`SwiftSft` 这类 pipeline 才是运行时总装线。它先准备模型和 processor，再准备模板，随后加载并编码数据，最后通过 `TrainerFactory.get\_trainer\_cls(args)` 选择 Trainer。

阶段	源码对象	用户可控参数	常见风险
模型准备	<code>`args.get_model_processor()`</code>	<code>`--model`、`--model_type`、`--torch_dtype`、`--device_map`、`--model_kwargs`</code>	模型类型自动匹配失败；训练和推理时模型特定参数不一致。
模板准备	<code>`args.get_template(processor)`</code>	<code>`--template`、`--system`、`--loss_scale`、`--agent_template`、`--max_length`</code>	模板不支持 packing / padding_free；base 模型误用 chat template。
数据准备	<code>`load_dataset`、 `_encode_dataset`、 `_post_process_datasets`</code>	<code>`--dataset`、`--columns`、`--split_dataset_ratio`、`--streaming`、 `--packing`、`--cached_dataset`</code>	字段映射错误；多模态路径不一致；流式训练未设置 <code>`max_steps`</code> 。
模型改造	<code>`TunerMixin.prepare_model`</code>	<code>`--tuner_type`、`--target_modules`、 `--lora_rank`、`--freeze_vit`、`--vit_lr`</code>	目标模块不匹配；QLoRA 后不能直接 merge；多模态子模块学习率混乱。
训练执行	<code>`trainer.train()`</code>	<code>`--learning_rate`、`--gradient_accumulation_steps`、`--deepspeed`、`--fsdp`、`--report_to`</code>	有效 batch size 与预期不一致；ZeRO/FSDP 与 checkpoint 策略不匹配。

命令与参数：一次运行如何被解释

ms-swift 的 CLI 设计把“命令名”与“Python 模块”解耦。`swift.cli.main` 维护路由表，读取命令名后找到相应模块，再根据分布式环境变量决定是否通过 `torch.distributed.run` 启动。

命令路由表

命令	模块	典型用途	是否可能 torchrun
`swift pt`	`swift.cli.pt`	CPT (继续预训练)。	是
`swift sft`	`swift.cli.sft`	SFT (监督微调)，最常用入口。	是
`swift rlhf`	`swift.cli.rlhf`	DPO、KTO、RM、PPO、GRPO、GKD 等后训练。	是
`swift infer`	`swift.cli.infer`	交互式或批量推理。	是
`swift deploy`	`swift.cli.deploy`	启动 OpenAI 风格服务。	通常否
`swift rollout`	`swift.cli.rollout`	RL 外部 vLLM rollout 服务。	通常否
`swift export`	`swift.cli.export`	Merge LoRA、量化、推送模型、缓存数据集。	通常否
`swift eval`	`swift.cli.eval`	调用 EvalScope 评测。	通常否
`swift sample`	`swift.cli.sample`	采样、蒸馏、PRM / ORM 过滤。	通常否
`swift web-ui`	`swift.cli.web-ui`	Gradio Web-UI。	否
`megatron sft`	`swift.cli._megatron.sft`	Megatron-SWIFT SFT。	是
`megatron export`	`swift.cli._megatron.export`	Megatron / HF 权重转换与导出。	是

YAML / JSON 与环境变量

ms-swift 允许把训练命令写成 YAML 或 JSON。`parse\_yaml\_args` 会把配置文件中的键值转成 CLI 参数，并把 `ENV` 字段写入当前进程环境变量。这一点对多模态模型尤其重要，因为部分模型的帧数、像素、音频参数既可以来自 `--model\_kwargs`，也可能来自环境变量。

```
ENV:
  MAX_PIXELS: "1003520"
  FPS_MAX_FRAMES: "12"
model: Qwen/Qwen3-4B-Instruct-2507
dataset:
  - AI-ModelScope/alpaca-gpt4-data-zh#500
  - swift/self-cognition#500
tuner_type: lora
learning_rate: 1e-4
output_dir: output/qwen3_sft
```

参数的四层分类

官方文档把命令行参数分为基本参数、原子参数、集成参数和特定模型参数。掌握这个分类可以避免“看到参数就堆”的坏习惯。集成参数是用户实际使用的命令参数；原子参数来自训练、数据、模板、生成、量化等局部域；基本参数跨任务共享；特定模型参数则需要回到模型官方实现或`model\_kwargs`解释。

参数域	代表参数	应先问的问题
模型	`model`、`model_type`、`torch_dtype`、`attn_impl`、`device_map`	模型能否被自动识别？训练和推理的 dtype、attention、特殊模型参数是否一致？
数据	`dataset`、`columns`、`split_dataset_ratio`、`dataset_num_proc`、`cached_dataset`	样本是否能被 AutoPreprocessor 正确转成标准格式？验证集是否独立？
模板	`template`、`system`、`max_length`、`loss_scale`、`agent_template`	哪些 token 参与损失？多轮、工具调用、多模态 token 是否符合模型协议？
训练	`learning_rate`、`per_device_train_batch_size`、`gradient_accumulation_steps`、`deepspeed`、`fsdp`	有效 batch size 是多少？显存与吞吐的取舍如何？
轻量微调	`tuner_type`、`target_modules`、`lora_rank`、`lora_alpha`、`freeze_vit`	只训练 LLM，还是同时训练 ViT / aligner？目标模块能否匹配？
RL	`rlhf_type`、`beta`、`reward_funcs`、`use_vllm`、`vllm_mode`	数据提供偏好对、label、prompt，还是 reward ground truth？采样和训练是否共卡？
生成与部署	`infer_backend`、`temperature`、`max_new_tokens`、`stream`、`merge_lora`	验证效果要确定性，还是要接近生产采样？LoRA 是否需要合并后再部署？

参数心法

新手只调模型、数据、batch、学习率、LoRA 和输出目录；中级用户再调模板、packing、评测、导出；高级用户才调分布式、RL rollout、Megatron 并行和插件扩展。

数据、模板与损失：训练样本如何变成梯度

大模型训练框架的关键问题不是“如何读 JSON”，而是如何把异构任务数据统一成可编码、可裁剪、可打包、可计算损失的样本。ms-swift 选择用标准数据集格式和 Template 编码器连接两端。

标准数据集格式

ms-swift 的标准格式以 ``messages`` 为核心，额外字段负责不同任务和模态：``rejected_response`` 用于 DPO / ORPO / CPO / SimPO / RM 等偏好任务；``label`` 用于 KTO 与分类；``images``、``videos``、``audios`` 存储多模态资源；``tools`` 用于 Agent；``objects`` 用于 grounding；``loss`` 与 ``loss_scale`` 则控制 token 是否参与训练以及权重。

字段	适用任务	语义	常见错误
<code>`messages`</code>	几乎所有任务	system / user / assistant / tool_call / tool_response 的对话序列。	把历史多轮写成字符串，导致模板无法识别轮次。
<code>`rejected_response`</code> / <code>`rejected_messages`</code>	DPO、ORPO、CPO、SimPO、RM	偏好对中的拒绝回答或完整拒绝消息序列。	多模态偏好对没有提供 rejected_images，导致图文不一致。
<code>`label`</code>	KTO、序列分类、回归	偏好标签、分类标签或回归值。	把 KTO 的布尔标签写成字符串。
<code>`images`</code> / <code>`videos`</code> / <code>`audios`</code>	多模态训练	文件路径、URL、base64 或视频帧列表。	相对路径在多机或容器里失效。
<code>`tools`</code>	Agent 训练	函数工具声明，配合 agent_template 格式化。	模型模板与工具调用格式不匹配。
<code>`loss`</code>	SFT / Agent 等	控制某个 assistant 段是否参与损失。	误以为能对 user token 生效；官方逻辑只对 assistant 相关段有意义。
<code>`loss_scale`</code>	SFT / Agent / RLHF 部分场景	更细粒度的 token 权重控制。	出现大于 1 的权重却未关闭二值 loss_scale 假设。

四类自动预处理入口

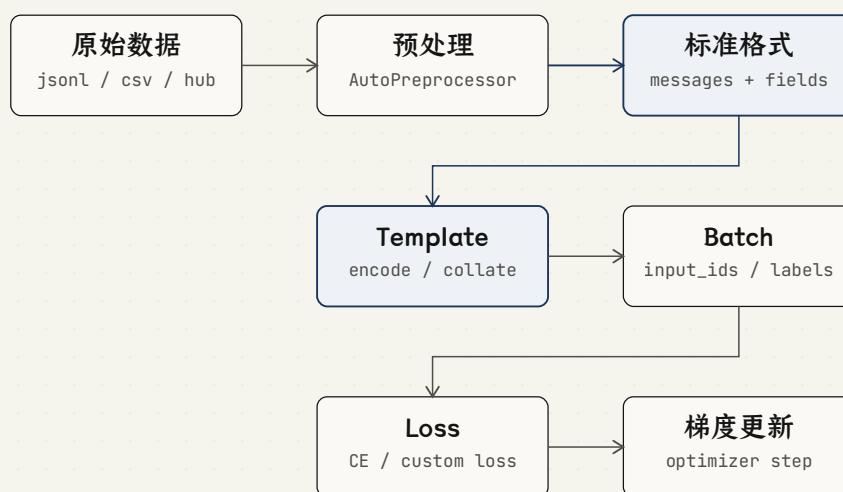
`AutoPreprocessor` 能处理 messages、ShareGPT、query-response、alpaca 四类常见格式。对新用户来说，优先把自己的数据转成这四类之一，再通过 `--columns` 做列名映射。只有当你需要自定义复杂字段、特殊采样或数据动态生成时，才应注册自定义数据集。

```
{
  "messages": [
    { "role": "user", "content": "浙江的省会在哪?" },
    { "role": "assistant", "content": "浙江的省会在杭州。" }
  ],
  "instruction": "翻译成英文",
  "input": "训练框架",
  "output": "training framework",
  "query": "1+1 等于几?",
  "response": "等于 2",
  "system": "你是个数学助手"
}
```

Template 的真实职责

Template 不是“把 prompt 拼成字符串”这么简单。它决定 system 的优先级、每轮对话的分隔符、assistant 部分的 label、工具调用如何编码、多模态占位符如何替换、哪些 token 被置为 `-100`、是否支持 padding_free / packing。对于训练框架来说，Template 是数据语义进入梯度的闸门。

FIGURE 3 DATA TO GRADIENT



数据进入训练前至少经历格式转换、模板编码、batch collate、loss mask 四次语义裁决。

loss 与 loss_scale 的区别

`loss` 是开关：某段 assistant 内容是否参与损失。`loss\_scale` 是权重：参与损失的 token 以什么权重进入平均。SFT 默认主要训练 assistant；RLHF 默认只关注最后一轮 response；预训练默认是 `all`。Agent 训练中工具调用可能需要更高权重，空 thinking 段可能需要忽略，这正是 `loss\_scale` 存在的原因。如果训练后模型学会了重复 system、复读用户问题或工具调用格式错乱，第一反应不要调学习率；先用 `get\_template(...).encode(data)` 打印 `input\_ids`、`labels` 和 `loss\_scale`，确认被训练的 token 是否正是你想训练的内容。

CPT / SFT 主链路：最常用训练路径

CPT (Continual Pre-Training, 继续预训练) 与 SFT (Supervised Fine-Tuning, 监督微调) 是 ms-swift 的基础训练路径。前者让模型继续吸收领域语料, 后者让模型学习对话、指令、格式和任务行为。多数企业项目会先做数据清洗, 再做 SFT; 只有领域知识缺口明显、语料足够大时才优先 CPT。

CPT 与 SFT 的差异

维度	CPT: `swift pt`	SFT: `swift sft`
目标	让模型继续学习领域语言分布、术语、文档风格和背景知识。	让模型在给定输入下输出期望回答、格式、推理过程或工具调用。
模板	默认 `use_chat_template=false`, 生成式模板; 官方说明可近似理解为 `sft --use_chat_template false --loss_scale all`。	默认使用 chat template, 通常只训练 assistant 内容。
数据	大段文本、文档、代码、领域语料。	messages、多轮对话、指令-答案、图文问答、工具调用轨迹。
风险	数据质量差会损害通用能力; 训练量不足时收益不明显。	模板和 loss mask 错误会直接教坏输出格式; 过拟合小数据会降低泛化。
评估	困惑度、领域问答、知识覆盖、下游任务。	验证集生成质量、格式遵循、任务指标、人工抽检。

单卡 LoRA SFT 最小闭环

```
CUDA_VISIBLE_DEVICES=0 \
swift sft \
  --model Qwen/Qwen3-4B-Instruct-2507 \
  --tuner_type lora \
  --dataset data/train.jsonl \
  --torch_dtype bfloat16 \
  --learning_rate 1e-4 \
  --lora_rank 8 \
  --lora_alpha 32 \
  --target_modules all-linear \
  --per_device_train_batch_size 1 \
  --gradient_accumulation_steps 16 \
  --max_length 2048 \
  --split_dataset_ratio 0.01 \
  --eval_steps 100 \
  --save_steps 100 \
  --output_dir output/qwen3_lora_sft
```

这条命令背后有三个关键数字：单卡 micro batch 为 1；梯度累积为 16；如果 world size 为 1，则有效 batch size 为 16。学习率 `1e-4` 是 LoRA 训练常见起点；全参数训练通常从更低学习率开始。`max\_length` 是防止训练 OOM 的第一道闸门，但它会删除或裁剪超长样本，因此必须配合训练前数据长度统计。

训练后推理、合并与发布

```
# 使用 LoRA adapter 推理
swift infer \
  --adapters output/qwen3_lora_sft/${RUN_ID}/${CHECKPOINT} \
  --stream true \
  --temperature 0

# 合并 LoRA 后用 vLLM 验证
swift infer \
  --adapters output/qwen3_lora_sft/${RUN_ID}/${CHECKPOINT} \
  --merge_lora true \
  --infer_backend vllm \
  --vllm_max_model_len 8192 \
  --temperature 0

# 推送到 ModelScope 或 Hugging Face
swift export \
  --adapters output/qwen3_lora_sft/${RUN_ID}/${CHECKPOINT} \
  --push_to_hub true \
  --hub_model_id your-org/your-model \
  --use_hf false
```

SFT 训练检查表

阶段	必须检查	失败信号	修正动作
训练前	数据样本能否被`get_template(...).encode`正确编码。	labels 全是`-100`或system/user 被训练。	修正数据格式、template、loss_scale。
训练前	长度分布与`max_length`的关系。	大量样本被 delete / truncate。	提高 max_length、切分样本、使用长上下文策略。
训练中	loss 曲线、学习率曲线、grad_norm、eval loss。	loss 突然 NaN；eval 不下降。	降学习率、检查数据异常、关高风险 kernel、换 dtype。
训练后	使用训练参数自动加载 adapter 推理。	推理提示模板与训练不一致。	保留`args.json`，必要时设置`--load_args false`并显式指定参数。
发布前	merge 后模型与 adapter 模式输出一致性。	merge 后效果明显变化。	检查 QLoRA 限制、target modules、dtype、特殊 token 保存。

轻量微调与显存工程：LoRA、QLoRA、packing

训练大模型时，算法选择往往受显存约束。ms-swift 暴露了三类手段：减少可训练参数、减少激活与中间状态、提高 token 利用率。不要把它们混为一谈。

三类显存策略

策略	代表参数	节省对象	代价
参数高效微调	<code>--tuner_type lora/adalora/adapter/reft/...</code>	可训练参数、优化器状态。	表达能力受 rank、目标模块和任务复杂度影响。
量化加载 / QLoRA	<code>--quant_method bnb`、GPTQ / AWQ 量化模型</code>	模型权重显存。	QLoRA 模型通常不能直接 Merge LoRA，部署链路受限。
激活节省	<code>--gradient_checkpointing true`、DeepSpeed、FSDP、sequence parallel</code>	激活、梯度、优化器状态、参数分片。	训练速度下降或通信复杂度上升。
token 利用率	<code>--packing true`、--padding_free true`</code>	padding 浪费与 batch 内空位。	packing 需要额外预处理；padding_free 依赖 attention 实现支持。
多模态子模块冻结	<code>--freeze_vit`、--freeze_aligner`、--vit_lr`、--aligner_lr`</code>	ViT、aligner 或 LLM 的训练状态。	错误冻结会让视觉或跨模态对齐能力不适配任务。

LoRA 决策表

任务情形	建议	理由
单卡或少卡，目标是格式、口吻、工具调用、小规模领域任务。	LoRA, `target_modules=all-linear`, rank 从 8 或 16 起。	成本低，输出可 merge，适合快速迭代。
数据量较大，目标是深层能力变化或领域知识吸收。	全参数或 LoRA + 更高 rank，必要时先 CPT 后 SFT。	仅低 rank adapter 可能不足以承载知识分布变化。
显存非常紧张，只需验证可行性。	QLoRA 或量化模型训练。	能显著降低权重显存，但后续 merge 和推理后端要提前确认。
多模态任务只改变语言输出，不改变视觉识别。	冻结 ViT / aligner，仅训练 LLM LoRA。	减少显存和过拟合风险。
多模态任务改变 OCR、grounding、细粒度视觉理解。	考虑 `lora_llm`、解冻 ViT / aligner，并给不同学习率。	视觉编码和跨模态投影本身需要适配。

packing 与 padding_free

packing 把多个短样本拼到接近 `max\_length`，减少 padding，通常能提高吞吐并稳定显存；padding_free 则把 batch 展平，减少 padding 计算，但不需要额外预处理。官方文档提示 padding_free 需结合 flash attention 与较新 Transformers 支持。选择时先看数据长度分布：短样本多、长度差异大，packing 收益高；数据预处理很贵或动态样本多，padding_free 更灵活。

显存优化顺序建议：先降低 `max\_length` 或清洗超长异常样本；再启用 gradient checkpointing；再调整 batch 与累积；再使用 LoRA / QLoRA；最后才引入 DeepSpeed ZeRO、FSDP、sequence parallel 或 Megatron。过早引入分布式会把数据和模板错误伪装成通信问题。

RL 后训练：DPO、RM、GRPO、GKD 与 rollout

RLHF (Reinforcement Learning from Human Feedback, 基于反馈的强化学习) 在 ms-swift 中不是一个算法, 而是一组训练任务。它们共享模型、模板、数据和 Trainer 基座, 但数据格式、参考模型、生成方式、奖励来源和优化目标不同。

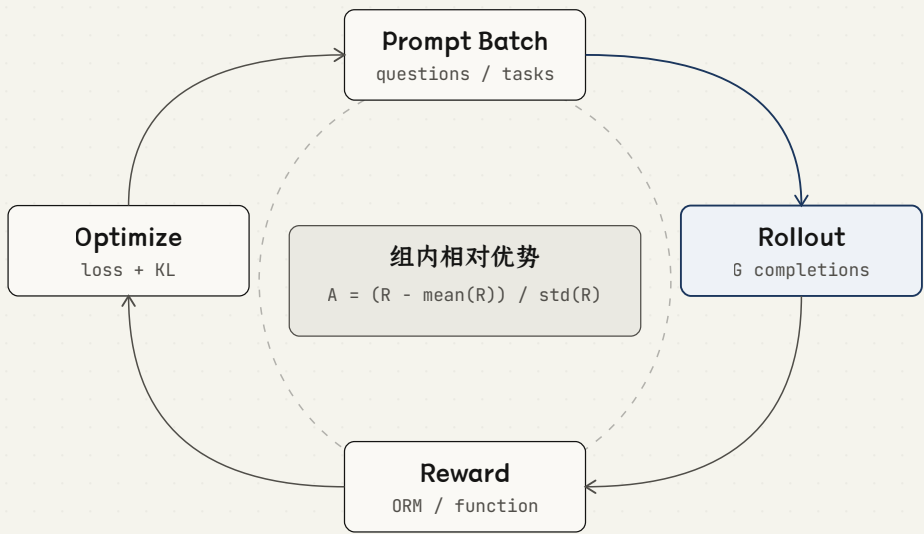
后训练任务地图

任务	数据需求	核心对象	适用场景
DPO	chosen / rejected 偏好对。	policy model、reference model、偏好损失。	有明确偏好数据, 希望不用在线 rollout。
KTO	单样本好坏标签 `label`。	可取 / 不可取回答的非成对偏好学习。	偏好对成本高, 但能标出单条回答好坏。
RM	偏好对或带 margin 的偏好数据。	奖励模型。	需要先训练 reward model, 再服务 PPO / rerank / filtering。
ORPO / SimPO / CPO	偏好对。	不同偏好损失变体。	希望简化 reference 或调整偏好目标。
PPO	prompt + reward / value 相关模型。	policy、ref、reward、value。	经典 RLHF, 但工程复杂度高。
GRPO	prompt + 可计算奖励字段。	组内采样、奖励函数、优势估计、KL。	数学、代码、可验证任务、多轮 Agent、RLVR。
GKD / OPSD	教师输出或 prompt。	teacher model / teacher server、student model。	蒸馏强模型行为, 或 on-policy 自蒸馏。

GRPO 的运行逻辑

GRPO (Group Relative Policy Optimization, 组相对策略优化) 对同一个 prompt 采样多个 completion, 用奖励函数给每个 completion 打分, 再用组内均值和标准差归一化出 advantage。相较 PPO, 它不依赖独立 value model, 而是在策略损失中加入参考模型 KL 惩罚。ms-swift 官方文档把 GRPO 集成到 `swift rlhf --rlhf\_type grpo` 与 Megatron-SWIFT 中, 并支持 colocate 与 server 两类 vLLM 模式。

FIGURE 4 GRPO TRAINING LOOP



GRPO 的复杂度在于训练循环与推理采样循环被绑定，工程风险常出现在 rollout、奖励和权重同步。

colocate 与 server 模式

模式	资源关系	优点	风险与调参
colocate	训练与 vLLM 推理共享 GPU。	部署简单，适合少量机器快速试验。	OOM 风险高；需要调 <code>`vllm_gpu_memory_utilization`</code> 、 <code>`sleep_level`</code> 、 <code>`offload_optimizer`</code> 、 <code>`offload_model`</code> 、 <code>`move_model_batches`</code> 。
server	训练与推理资源分离，通过 <code>`swift rollout`</code> 启外部 vLLM。	更接近生产，采样资源可独立扩展。	需要处理网络、超时、TP/DP、服务异常和权重同步延迟。

奖励函数与额外字段

GRPO 数据可以只提供 prompt，但可验证任务通常还要提供 ``solution``、``answer``、测试用例或环境状态。官方文档强调 GRPO 会把额外字段透传给 ORM，因此自定义奖励函数可以从样本额外字段读取 ground truth。这个设计非常重要：奖励函数不应从 prompt 文本里重新解析标准答案，否则数据清洗和奖励逻辑会耦合得难以复现。

做 GRPO 前先完成三个小实验：一是单 prompt 采样，确认答案多样性；二是离线奖励函数单测，确认奖励均值和标准差不是常数；三是小 batch 训练，检查 ``reward_std``、``frac_reward_zero_std``、``kl``、``clip_ratio`` 等指标。缺任何一项都不要上多机长跑。

Megatron-SWIFT: 大模型并行训练路线

Megatron-SWIFT 把 Megatron 的并行技术接入 ms-swift 的数据、模板和任务协议。它适合全参数大模型、MoE、长上下文、FP8、RL 大规模采样等场景。与普通 `swift sft` 相比，它的学习成本来自并行维度和 checkpoint 格式。

支持能力

官方 Megatron-SWIFT 文档列出预训练、SFT、GRPO、GKD、DPO、KTO、RM、Embedding、Reranker、序列分类等任务，并覆盖全参数、LoRA、MoE、多模态与 FP8。README 中还明确提到 TP、PP、SP、CP、ETP、EP、VPP 等并行策略。不要把这些缩写当作参数装饰，它们决定模型、序列、上下文、专家和流水线如何切分。

并行策略	含义	什么时候考虑	主要代价
DP	Data Parallel, 数据并行。	模型单卡能放下，希望提高吞吐。	显存占用高，参数复制。
TP	Tensor Parallel, 张量并行。	单层矩阵太大或 MoE / 大模型需切张量。	通信频繁，尽量放在高速互联域内。
PP / VPP	Pipeline Parallel / Virtual PP, 流水线并行。	跨节点大模型训练，层级可切分。	pipeline bubble、调度复杂。
SP	Sequence Parallel, 序列并行。	长上下文训练，降低序列维度激活压力。	依赖 attention / loss 处理支持。
CP	Context Parallel, 上下文并行。	极长上下文或 Ring-Attention 场景。	通信与实现约束高。
EP / ETP	Expert Parallel / Expert Tensor Parallel, 专家并行。	MoE 模型专家层训练。	专家通信、负载均衡与 capacity 设置复杂。

Mcore-Bridge 的意义

早期 Megatron 训练常见痛点是权重格式转换：HF safetensors 到 Megatron，再从 Megatron 回到 HF。Mcore-Bridge 的目标是让 Megatron 训练尽量像 Transformers 一样加载和保存 safetensors，支持 LoRA 增量权重转换、Megatron 到 vLLM 权重同步、超大模型多机转换。ms-swift 文档明确推荐 Mcore-Bridge 路线。

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
NPROC_PER_NODE=2 \
CUDA_VISIBLE_DEVICES=0,1 \
megatron sft \
  --model Qwen/Qwen2.5-7B-Instruct \
  --save_safetensors true \
  --dataset data/train.jsonl \
  --tensor_model_parallel_size 2 \
  --sequence_parallel true \
  --micro_batch_size 16 \
  --global_batch_size 16 \
  --finetune true \
  --cross_entropy_loss_fusion true \
  --lr 1e-5 \
  --num_train_epochs 1 \
  --output_dir megatron_output/qwen_sft \
  --max_length 2048
```

何时不该用 Megatron-SWIFT

- 模型 7B 或更小，LoRA SFT 已能满足，且训练时间不是瓶颈。
- 你还没确认数据、模板、loss mask、验证集和导出闭环。
- 团队没有排查分布式通信、checkpoint 恢复、CUDA/NPU 环境差异的能力。
- 任务目标不是吞吐、上下文长度、MoE 或全参数训练，而只是格式对齐。

规模化原则

先用 `swift sft` 证明任务、数据和目标函数成立；再用 Megatron-SWIFT 扩规模。把“算法正确性”和“系统吞吐”分开验证，是训练工程最基本的风险控制。

多模态、Agent 与专门任务

多模态和 Agent 是 ms-swift 相对突出的能力域。它们的共同点是：训练样本不再只有文本 response，而是包含图片、视频、音频、工具定义、工具调用、工具返回、bbox 或其他结构化字段。框架价值在于把这些结构映射到统一模板和 loss 规则。

多模态训练

多模态样本在标准 messages 基础上增加 ``images``、``videos``、``audios``。正文中用 `<image>`、`<video>`、`<audio>` 标记模态资源插入位置。Template 会把这些特殊 token 交给对应 processor 处理。多模态训练的主要风险不在模型，而在资源路径、像素/帧数控制、bbox 坐标和子模块冻结策略。

问题	参数 / 字段	判断
图片过大导致 OOM	<code>`--max_pixels`</code> 、模型环境变量如 <code>`MAX_PIXELS`</code>	训练和推理的缩放策略必须一致，grounding 任务尤其敏感。
视频训练太慢	<code>`FPS_MAX_FRAMES`</code> 、 <code>`VIDEO_MAX_PIXELS`</code> 、样本级控制字段	先限制帧数，再考虑数据采样；不要盲目增 batch。
只想改变回答格式	<code>`--freeze_vit true`</code> 、 <code>`--freeze_aligner true`</code>	通常冻结视觉侧，训练 LLM adapter 即可。
OCR / grounding 能力不足	<code>`--freeze_vit false`</code> 、 <code>`--vit_lr`</code> 、 <code>`objects`</code> 、 <code>`norm_bbox`</code>	需要谨慎解冻视觉或 aligner，并验证坐标归一化。
多模态 packing	<code>`--packing true`</code>	可提高训练速度，但要确认模板支持和资源读取稳定性。

Agent 训练

Agent 训练的难点是工具协议。不同模型对 `tools`、`tool_call`、`tool_response` 的序列化方式不同。ms-swift 通过 ``agent_template`` 把统一 Agent 数据集映射到各模型格式。一个 AgentTemplate 需要实现工具列表格式化、工具调用格式化、工具返回格式化和部署时 `toolcall` 解析。

```
{
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "search",
        "parameters": {
          "type": "object"
        }
      }
    }
  ],
  "messages": [
    {
      "role": "user",
      "content": "查找杭州今天的天气"
    },
    {
      "role": "tool_call",
      "content": {
        "name": "search",
        "arguments": {
          "q": "杭州 天气"
        }
      }
    },
    {
      "role": "tool_response",
      "content": {
        "weather": "晴"
      }
    },
    {
      "role": "assistant",
      "content": "杭州今天晴。"
    }
  ]
}
```

Embedding、Reranker 与序列分类

这三类任务与 causal LM 不同，label 可能是样本级而不是 token 级。源码中 Trainer 对 sequence parallel 输入做了额外处理，避免把一维 labels 当作 token labels。使用这些任务时，不要沿用 SFT 的评价方式；Embedding 看向量检索指标，Reranker 看排序损失和列表级指标，序列分类看分类或回归指标。

| 评测、推理、导出与部署

训练框架是否可用，最终看训练后能否被评测、推理、导出和部署复用。ms-swift 的全链路价值在这里体现：训练参数会保存到 checkpoint，推理可自动读取 adapter 中的 `args.json`，导出可以 merge、量化、推送，部署可以走 OpenAI 风格接口。

训练后的四个问题

问题	命令	关键参数	验收标准
模型是否学到了任务？	`swift infer`	`--adapters` 或 `--model`、`--temperature 0`、`--load_data_args true`	固定验证 prompt 输出稳定，格式正确。
是否能批量评测？	`swift eval`	`--eval_dataset`、EvalScope 参数	指标可复现，评测集和训练集无泄漏。
是否能部署？	`swift deploy`	`--infer_backend vllm/sglang/lmdeploy`、`--merge_lora`	OpenAI 客户端可调用，延迟和吞吐达标。
是否能发布？	`swift export`	`--merge_lora`、`--quant_method`、`--push_to_hub`	权重、tokenizer、配置、额外文件完整。

adapter、full checkpoint 与 merge 的区别

`--model` 指向完整权重目录，里面应包含模型、tokenizer、config 等完整文件；`--adapters` 指向增量 adapter 权重目录，通常含 `adapter\_model.safetensors` 和训练参数 `args.json`。LoRA 推理可以直接挂 adapter，也可以 merge 成完整权重。生产部署通常倾向 merge 后再量化或使用推理后端，但 QLoRA、MoE、Megatron 格式等场景需要单独验证兼容性。

量化导出

ms-swift 支持 AWQ、GPTQ、FP8、BNB 等量化导出；README 也说明导出的模型可配合 vLLM、SGLang、LMDeploy 推理加速。量化不应只看显存下降，还要看目标后端、上下文长度、吞吐、精度回归和模型结构支持。尤其是 FP8 与 Megatron-SWIFT 相关能力，应按对应版本和硬件栈核对。

发布前最小验收：同一验证集上比较三种输出，训练中 checkpoint 推理、merge 后权重推理、目标部署后端推理。三者必须在关键格式和任务指标上等价，否则不能把训练成功等同于上线成功。

扩展、排障与掌控清单

从会用到掌控的分水岭，是能否新增模型、数据、损失或奖励函数，并能定位训练失败发生在哪一层。ms-swift 的扩展点很清晰，排障也应按层推进。

扩展点总表

扩展点	位置	何时使用	最小测试
自定义模型	`swift/model`, 外部 `plugin.py`	新模型不能自动识别, 或加载逻辑特殊。	注册 <code>model_type</code> , 加载模型, 单 <code>prompt</code> 推理对齐官方实现。
自定义模板	`swift/template`	模型 <code>chat</code> 格式、工具调用、多模态 <code>token</code> 不同。	打印 <code>encode</code> 后 <code>input_ids</code> / <code>labels</code> , 与官方 <code>prompt</code> 对齐。
自定义数据集	`dataset_info.json` 或外部注册	标准格式不足以表达采样和预处理。	加载前 10 条, 验证字段、长度、 <code>loss mask</code> 。
自定义 <code>loss</code>	`swift/loss`	SFT / <code>pretrain</code> / <code>reranker</code> / <code>embedding</code> 需要新损失。	单 <code>batch</code> 前向后向, <code>loss</code> 为标量且梯度非零。
<code>loss_scale</code>	`swift/loss_scale`	工具调用、 <code>thinking</code> 、分段答案需要不同权重。	可视化每段 <code>token</code> 权重。
<code>metrics</code>	`swift/metrics`	验证指标不只是 <code>loss</code> 。	小验证集指标与独立脚本一致。
<code>callbacks</code>	`swift/callbacks`	保存、早停、日志、外部系统联动。	模拟训练开始、保存、结束事件。
<code>optimizer</code>	`swift/optimizers`	自定义优化器或不同子模块学习率。	<code>param_groups</code> 正确, 学习率日志可解释。
<code>tuner</code>	`swift/tuner_plugin`	不满足内置 <code>LoRA</code> / <code>adapter</code> 逻辑。	<code>prepare</code> 、 <code>save</code> 、 <code>from_pretrained</code> 三段闭环。
ORM / PRM	`swift/rewards`	GRPO 或采样过滤需要结果/过程奖励。	离线样本奖励分布合理, 异常返回可控。

分层排障法

症状	优先排查层	诊断动作
训练一开始就 OOM	长度、batch、dtype、模型加载。	统计 token 长度；降低 `max_length`；确认 dtype；关掉不必要的解冻模块。
loss 不下降	数据、模板、标签、学习率。	打印 labels；抽检样本质量；降低学习率；检查 assistant 段是否被训练。
模型输出模板符号	template 与 tokenizer。	确认 base/chat 模型模板，检查特殊 token 是否保存。
LoRA merge 后效果变差	量化、target modules、dtype、保存文件。	先用 adapter 模式验证；检查 QLoRA 限制；确认 additional_saved_files。
多模态训练很慢	资源读取、图像大小、dataset_num_proc。	改绝对路径；限制像素和帧数；观察 CPU 利用率与 I/O。
GRPO reward_std 为 0	采样和奖励函数。	提高采样多样性；检查 reward 解析；确认 ground truth 字段透传。
多机训练卡住	缓存、随机性、通信、共享存储。	设置共享 `MODELSCOPE_CACHE`；固定数据切分；检查 NCCL/HCCCL 日志。
vLLM rollout OOM	colocate 显存分配与同步。	调 `sleep_level`、offload、TP、`move_model_batches`、仅同步 LoRA。

从 0 到掌控的能力阶梯

1. 能跑通：完成官方 quick start，知道 checkpoint、adapter、args.json 在哪里。
2. 能换数据：把自己的 JSONL 转成标准格式，能解释每个字段为什么存在。
3. 能看模板：打印 input_ids / labels，确认模型学的是 assistant 而不是 prompt。
4. 能控显存：用 batch、累积、max_length、LoRA、packing、checkpointing 形成稳定配置。
5. 能闭环：训练后能 infer、eval、merge、quantize、deploy，并比较输出一致性。
6. 能对齐：会选择 DPO / KTO / RM / GRPO / GKD，并能写奖励函数或偏好数据。
7. 能扩展：注册新模型、新模板、新数据集或新 reward，并补最小测试。
8. 能扩规模：理解 DeepSpeed、FSDP、sequence parallel、Megatron 并行和 Mcore-Bridge 的取舍。

最终判断

掌握 ms-swift 的标志不是记住参数，而是能把一个训练需求分解为：模型注册是否满足、数据标准格式如何写、模板如何编码、损失训练哪些 token、Trainer 走哪条路径、显存如何预算、评测如何闭环、产物如何上线。

附录：术语、命令与来源

A. 常用命令速查

目标	命令骨架	备注
继续预训练	`swift pt --model ... --dataset ...`	默认生成式模板与 `loss_scale=all` 语义。
监督微调	`swift sft --model ... --dataset ... --tuner_type lora`	最常用训练入口。
偏好优化	`swift rlhf --rlhf_type dpo --dataset ...`	需要 chosen / rejected 数据。
GRPO	`swift rlhf --rlhf_type grpo --use_vllm true`	需要 prompt、奖励函数和 rollout 配置。
推理	`swift infer --adapters ... --stream true`	adapter 中的 `args.json` 会被自动读取。
部署	`swift deploy --model ... --infer_backend vllm`	服务化前先用 infer 验证。
导出	`swift export --adapters ... --merge_lora true`	用于 merge、量化和推送。
M-训练	`megatron sft --model ... --tensor_model_parallel_size ...`	适合大规模并行。
M-导出	`megatron export --to_hf true ...`	MoE / Megatron 权重不要盲目用普通 export。

B. 术语表

CPT: Continual Pre-Training, 继续预训练, 使用领域语料继续训练语言建模目标。

SFT: Supervised Fine-Tuning, 监督微调, 使用输入输出样本训练模型遵循指令。

LoRA: Low-Rank Adaptation, 低秩适配, 只训练附加低秩矩阵以降低训练成本。

QLoRA: 在量化权重上训练 LoRA, 降低权重显存, 但导出和推理兼容性需要验证。

Template: 把 messages、工具调用和多模态资源转成模型 token 序列的编码协议。

loss_scale: token 级损失权重控制, 用于忽略或强调特定输出片段。

ORM: Outcome Reward Model / 结果奖励函数, 按最终回答给奖励。

PRM: Process Reward Model / 过程奖励函数, 按推理过程或步骤给奖励。

rollout: RL 训练中由当前策略生成 completion 的采样过程。

Mcore-Bridge: ModelScope 的 Megatron-Core 桥接工具, 降低 safetensors 与 Megatron 格式转换成本。

C. 关键来源

核心事实来自 modelscope/ms-swift GitHub README 与本地源码快照 commit `7dd6b0e`、PyPI `ms-swift 4.2.1`、GitHub Releases `v4.2.1` / `v4.2.0`、ReadTheDocs latest `swift 4.3.0.dev0`、AAAI 2025 论文与 arXiv:2408.05517。完整来源、文件路径和边界说明见 `sources.md`。